Da Vinci
PRIMERA ESCUELA DE ARTE MULTIMEDIAL

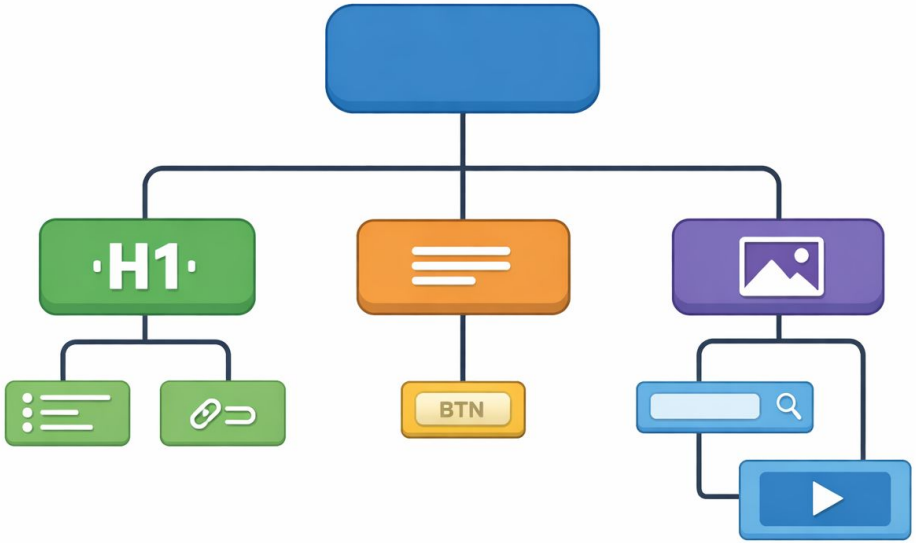
Clase 03

Analista de Sistemas

Materia:
Plataformas de Desarrollo

Profesor: Jonathan Cruz
Equipo docente:
- Jonathan Cruz
- Victor Villafañe

DOM



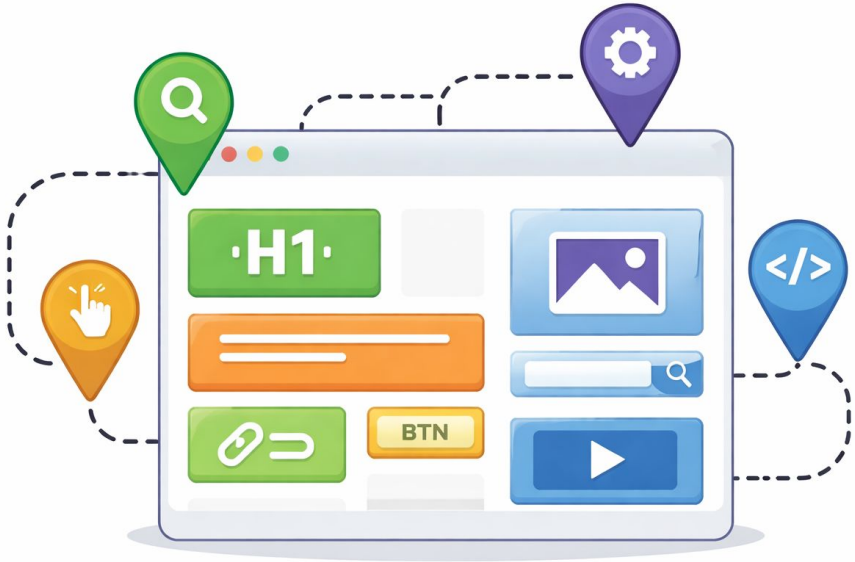
El DOM es un árbol de elementos HTML

El navegador transforma el HTML en una estructura que JavaScript puede modificar.

DOM

El DOM (Document Object Model) nos permite acceder y gestionar un documento HTML

El JavaScript Manipula el DOM



El DOM es como un mapa

- Nos permite ubicar elementos dentro de la página
- Podemos acceder a ellos usando selectores
- Funciona de forma similar a CSS

JavaScript usa el DOM como un mapa para encontrar elementos y trabajar con ellos

querySelector()

querySelector()

En Javascript existe un objeto llamado document

Con éste podemos gestionar un documento HTML a través de varios métodos, uno de ellos es **querySelector()**

El método en cuestión nos permite referenciar a un elemento por alguna característica propia del mismo como lo hace CSS, por ejemplo por su id

querySelector()

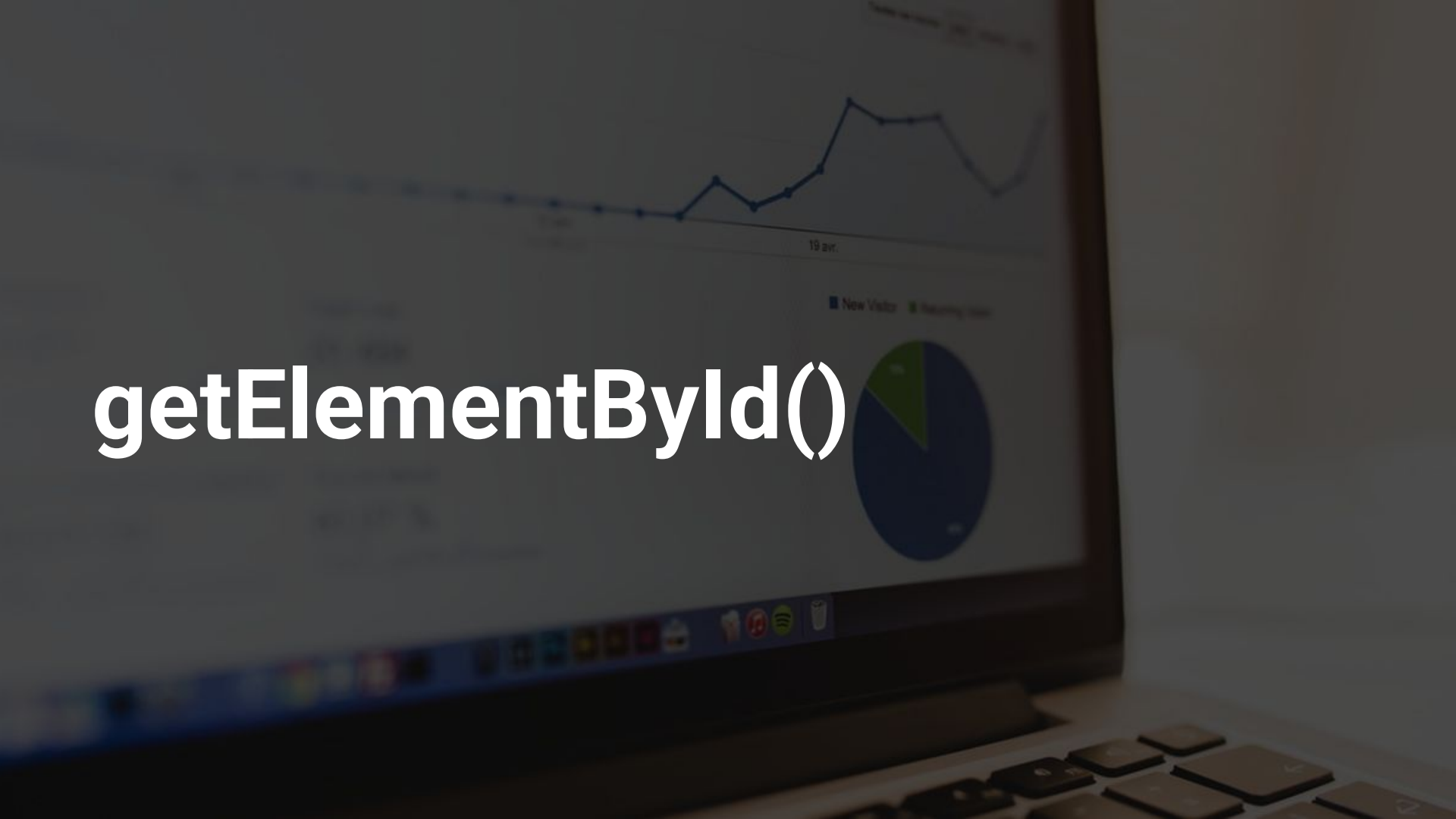
```
<!-- HTML -->
<h1>DOM</h1>
<p class="textoImportante">Texto</p>

<script>
    //JS
    const titulo = document.querySelector("h1");
    const texto = document.querySelector(".textoImportante");
</script>
```

Se usan Selectores como en CSS:

- Etiqueta
- Clase
- ID

getElementById()



getElementById()

Una alternativa a querySelector() es getElementById()

El método getElementById() también nos permite recuperar un elemento, aunque sólo por su id

El problema es que en muchas ocasiones tendremos que hacer referencias por otro tipo de característica como por ejemplo el nombre de una clase o el valor de un atributo

getElementById()

```
<!-- HTML -->
<input type="text" id="inputNombre" placeholder="Ingrese el nombre" />
<script>
    //JS
    const inputNombre = document.getElementById("inputNombre");
</script>
```

Manos a la obra

En un index.html

- Crear un <h1> con un título
- Seleccionar el H1
- Mostrarlo en consola

innerText e innerHTML

innerText e innerHTML

Dentro de un elemento HTML podemos ingresar texto

Para esto debemos usar la propiedad innerText como ya vimos

innerText e innerHTML

```
const mostrarNombre = () => {
    //Guardamos el nombre del usuario, a través del atributo value.
    let nombre = input_nombre.value;
    //Insertamos texto dentro del elemento respuesta_saludo.
    respuesta_saludo.innerText = nombre + " gracias por ingresar tu
nombre";
}
```

innerText e innerHTML

Una alternativa a esto, si quisiéramos ingresar código HTML en lugar de sólo texto, es innerHTML:

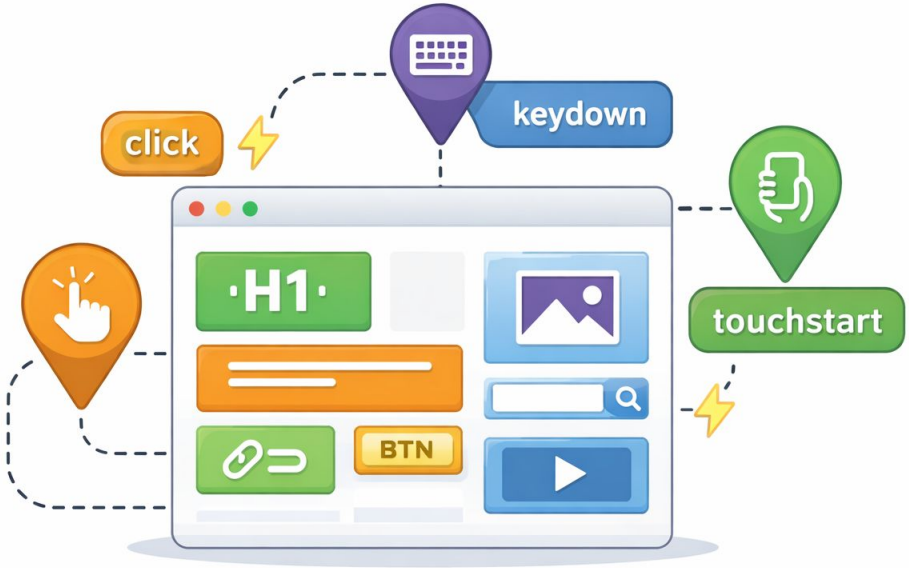
innerText e innerHTML

```
const mostrarNombre = () => {
    //Guardamos el nombre del usuario, a través del atributo value.
    let nombre = input_nombre.value;
    //Insertamos texto dentro del elemento respuesta_saludo.
    respuesta_saludo.innerHTML = "<strong>" + nombre + "</strong>" + "
gracias por ingresar tu nombre";
}
```

Manos a la obra

En un index.html

- Crear un <h1> con un título
- Seleccionar el H1
- Modificar el contenido del <H1> con innerText



El DOM es una interfaz interactiva

- Los elementos pueden reaccionar a acciones del usuario
- Podemos detectar eventos como clicks o escritura
- JavaScript define qué hacer cuando ocurre un evento

Los eventos permiten que la página responda a las acciones del usuario

Eventos

Eventos

Un evento es un bloque de código programado que va a ejecutarse en un momento dado

En Javascript existen muchos eventos, a continuación podemos ver varios de estos: https://www.w3schools.com/jsref/dom_obj_event.asp

Vamos a ver uno muy utilizado: click.

Eventos

```html
<!-- HTML -->
<button id="btn" type="button">Hace Click</button>
<script>
    // JS
    const boton = document.querySelector("#btn");

    boton.addEventListener("click", () => {
        console.log("Hiciste click");
    });
</script>
```

Antes solo leíamos el DOM…
Ahora reaccionamos a lo que hace el usuario.

Eventos

¿Qué cosas puede hacer un usuario en una página web?

Eventos

```
<!-- Tenemos un HTML con tres elementos -->
<form action="">
    <input type="text" id="inputNombre" placeholder="Ingresar nombre"
/>
    <button type="button" id="btnNombre" > Presentarse </button>
    <div id="respuestaSaludo"></div>
</form>
```

Eventos

```
//JS: Referenciamos a los tres elementos.
const inputNombre = document.querySelector("#inputNombre");
const btnNombre = document.querySelector("#btnNombre");
const respuestaSaludo = document.querySelector("#respuestaSaludo");
```

Eventos

```
const mostrarNombre = () => {
    //Guardamos el nombre del usuario a través del atributo value.
    let nombre = inputNombre.value;
    //Insertamos texto dentro del elemento respuesta_saludo.
    respuestaSaludo.innerText = nombre + " gracias por ingresar tu
nombre";
}
```

Eventos

```
//Asignamos al elemento btnNombre, que al llamar al evento click se
llame a la función mostrarNombre()
btnNombre.addEventListener("click", mostrarNombre);
```

Manos a la obra

En un index.html

- Crear un <h1> con un título
- Crear un <button>
- Seleccionar cada elemento
- Crear una función que cambie el contenido del H1
- Invocar la función cuando se presiona el <button>

Modo template

Modo template

Para ingresar varias líneas de código podemos utilizar el modo template

Modo template

```
const mostrarNombre = () => {
    //Guardamos el nombre del usuario, a través del atributo value.
    let nombre = inputNombre.value;
    //Insertamos texto dentro del elemento respuestaSaludo.
    respuestaSaludo.innerHTML = `
        <p> Hola, ${nombre} </p>
        <p> Gracias por registarte </p>
    `
}
```

Crear elementos

Crear elementos

El objeto document cuenta en su lista de métodos con createElement(), el cual nos permite crear un elemento html de forma dinámica

Crear elementos

```
//Crear un elemento de tipo <li>
let li = document.createElement("li");
```

Agregar elementos

Agregar elementos

Pero para que ese elemento esté disponible dentro de nuestra página tendremos que agregarlo dentro de otro elemento

Para eso debemos usar el método appendChild()

Agregar elementos

```html
<!-- HTML -->
<ul id="ul_lista">
    <!-- Contenido que se va a generar desde Javascript -->
</ul>
```

Agregar elementos

```
/* JS */
//Hacemos referencia al elemento con id ul_lista.
const ul_lista = document.querySelector("#ul_lista");
//Creamos un elemento de tipo <li>
let li = document.createElement("li");
//Introducimos texto
li.innerText = "Texto";
//Agregamos al elemento <ul> padre el elemento <li> hijo.
ul_lista.appendChild(li);
```

Ejercicio Final

En un index.html

- Crear un <input>, <button> y una <ul>
- Hacer que cada vez que el usuario escriba un nombre y haga click en el boton se ejecute una función addItem(*nombre*)
- La función agrega un <li> dentro de la <ul>

Eliminar elementos

Eliminar elementos

El proceso para eliminar un elemento de la página es similar a appendChild()

Primero debemos saber cuál es el elemento, y luego desde éste llamar a removeChild()

Eliminar elementos

```
ul_lista.removeChild(li);
```

Buen Codeo y que el Codeo te acompañe